

PAC-MAN GAME

A fully playable Pac-Man built from scratch in Java — no database, just Swing and pure OOP.

● Week 9 Complete

Java JDK 21

VS Code

Swing / JPanel

Sound + Pause Menu

WHAT WAS BUILT

Audio System

- ✓ SoundManager.java — new class wrapping javax.sound.sampled
- ✓ chomp, death, and eatGhost loaded as reusable Clips
- ✓ Looping siren starts/stops with the frightened window
- ✓ play() rewinds and restarts a clip so rapid re-triggers don't overlap
- ✓ Sound hooks wired into dot-eating, ghost-eating, and death events

Animation & Pause Menu

- ✓ PacMan sprite now rotates via Graphics2D.rotate() to face movement
- ✓ Fallback vector mouth toggles open/closed on an animation counter
- ✓ MENU game state added with a title screen and start prompt
- ✓ P / ESC toggles a PAUSED state mid-round without losing progress
- ✓ PAUSED overlay reuses drawOverlay() with a resume hint

KEY TECHNIQUES

Clip Rewind Instead of a Clip Pool

play() calls clip.stop(), clip.setFramePosition(0), then clip.start() on a Clip that was opened once in the constructor, rather than allocating a new Clip per playback. This keeps a fixed, small set of open audio lines and lets rapid back-to-back dot chomps restart cleanly instead of piling up overlapping clips.

Siren State Tied to the Frightened Timer, Not a New Flag

Rather than introducing a separate boolean to track whether the siren should be playing, run() checks the existing frightenedTimer > 0 condition every frame and calls startSiren() or stopSiren() accordingly. The siren can never drift out of sync with frightened mode because it's a direct function of state the game already maintains, not a second source of truth.

Rotation-Based Facing Instead of Four Sprites

PacMan.draw() computes a single angle from the current dx/dy and calls g2.rotate(angle, centerX, centerY) before drawing one sprite, so PacMan.png serves all four directions without needing separate up/down/left/right art. The same angle also feeds the fallback vector mouth's arc start angle, keeping the sprite and no-sprite rendering paths visually consistent.

Pause Freezes update(), Not repaint()

togglePause() only flips the state field. Inside run(), pacman.update() and ghost.update() are gated behind state == PLAYING, but paintComponent() keeps running every frame regardless of state. The result: the game visibly freezes mid-frame the instant P is pressed, with no separate pause-render path needed.

SOURCE CODE

```
public class SoundManager {
    private Map clips = new HashMap<>();
    private Clip sirenClip;

    public SoundManager() {
        load("chomp", "assets/sounds/chomp.wav");
        load("death", "assets/sounds/death.wav");
        load("eatGhost", "assets/sounds/eatghost.wav");
        loadSiren("assets/sounds/siren.wav");
    }

    private void load(String name, String path) {
        try {
            AudioInputStream stream = AudioSystem.getAudioInputStream(new File(path));
            Clip clip = AudioSystem.getClip();
            clip.open(stream);
            clips.put(name, clip);
        } catch (Exception e) {
            System.out.println("Could not load sound: " + path);
        }
    }

    public void play(String name) {
        Clip clip = clips.get(name);
        if (clip == null) return;
        clip.stop();
        clip.setFramePosition(0);
        clip.start();
    }

    public void startSiren() {
        if (sirenClip == null) return;
        if (!sirenClip.isRunning()) {
            sirenClip.setFramePosition(0);
            sirenClip.loop(Clip.LOOP_CONTINUOUSLY);
        }
    }

    public void stopSiren() {
        if (sirenClip != null) sirenClip.stop();
    }
}
```

```
// Siren only plays while frightened mode is active
if (frightenedTimer > 0) {
    soundManager.startSiren();
} else {
    soundManager.stopSiren();
}

int eaten = pacman.eatDot();
if (eaten == 2) {
    scoreManager.addDot();
    soundManager.play("chomp");
}
if (eaten == 3) {
    scoreManager.addPowerPellet();
    activateFrightenedMode();
}
```

GamePanel.java (excerpt — pause toggle)

java

```
private void togglePause() {
    if (state == GameState.PLAYING) {
        state = GameState.PAUSED;
        soundManager.stopSiren();
    } else if (state == GameState.PAUSED) {
        state = GameState.PLAYING;
    }
}

// In keyPressed():
if (code == KeyEvent.VK_P || code == KeyEvent.VK_ESCAPE) {
    togglePause();
    return;
}
```

PacMan.java (excerpt — rotation + chomp animation)

java

```
public void draw(Graphics g) {
    Graphics2D g2 = (Graphics2D) g.create();
    int size = Maze.TILE_SIZE;

    double angle = 0; // default: facing right
    if (dx == -1) angle = Math.PI;
    else if (dy == -1) angle = -Math.PI / 2;
    else if (dy == 1) angle = Math.PI / 2;

    if (sprite != null) {
        g2.rotate(angle, x + size / 2.0, y + size / 2.0);
        g2.drawImage(sprite, (int) x, (int) y, size, size, null);
    } else {
        g2.setColor(Color.YELLOW);
        if (animFrame == 0) {
            int startAngle = (int) Math.toDegrees(-angle) + 30;
            g2.fillArc((int) x, (int) y, size, size, startAngle, 300);
        } else {
            g2.fillOval((int) x, (int) y, size, size);
        }
    }
    g2.dispose();
}
```

App.java, GameWindow.java, Maze.java, ScoreManager.java, GameState.java, RedGhost.java, BrownGhost.java, BlueGhost.java, and YellowGhost.java are unchanged from Week 8.

10-WEEK PROGRESS

Week 1 — Setup & Window	✓
Week 2 — Maze Design	✓
Week 3 — Pac-Man Character	✓
Week 4 — Dot Collection & Scoring	✓
Week 5 — Ghost Base Class	✓
Week 6 — All Four Custom Ghosts	✓

Week 7 — Power Pellets & Frightened Mode	✓
Week 8 — Collisions, Lives & Game States	✓
Week 9 — Polish & Audio	✓
Week 10 — Levels & Final Build	0%

UP NEXT

W
10

Levels & Final Build

Add a Level 2 with faster ghosts, wire the main menu into a full replay loop, then package the game as a runnable .jar and write the README.

nextLevel() · ghost speed scaling · jar packaging · README.md